

# Lab 4: Polymorphism and abstract

## I. Polymorphism

**Polymorphism = "Many forms"**

In Java, polymorphism allows a **parent class reference to point to child class objects**, enabling dynamic behavior based on the actual object type at runtime.

There are two types:

1. **Compile-time (static):** Method overloading
2. **Runtime (dynamic):** Method overriding → **main focus here**

```
class Animal {  
    2 overrides  
    void makeSound() {  
        System.out.println("Some animal sound");  
    }  
}  
  
class Dog extends Animal {  
    @Override  
    void makeSound() {  
        System.out.println("Bark!");  
    }  
}  
  
class Cat extends Animal {  
    @Override  
    void makeSound() {  
        System.out.println("Meow!");  
    }  
}
```

```

public class Main {
    public static void main(String[] args) {
        Animal a1 = new Dog(); // Polymorphism
        Animal a2 = new Cat();

        a1.makeSound(); // Bark!
        a2.makeSound(); // Meow!

        // Use in arrays or loops
        Animal[] animals = {new Dog(), new Cat()};
        for (Animal a : animals) {
            a.makeSound(); // Dynamically resolves correct method
        }
    }
}

```

## When to Use Polymorphism?

- When you want to **treat different objects in the same way**.
- Helps in writing **extensible, clean, and decoupled code**.
- Often used in real-world frameworks (e.g., GUI components, animal behavior, data processors).

## II. Abstract

### 2.1 Abstract class

```

abstract class Vehicle {
    // Abstract method
    // 2 implementations
    abstract void startEngine();

    // Concrete method
    void fuelUp() {
        System.out.println("Fueling up the vehicle...");
    }

    // You can add properties
    String brand = "Generic Vehicle";
}

```

```

class Car extends Vehicle {
    @Override
    void startEngine() {
        System.out.println("Car engine started!");
    }
}

class Motorcycle extends Vehicle {
    @Override
    void startEngine() {
        System.out.println("Motorcycle engine started!");
    }
}

```

## 2.2 Interface

Java **does not support multiple inheritance with classes** to avoid ambiguity.

```

class A {
    2 overrides
    void sayHello() {
        System.out.println("Hello from A");
    }
}

class B extends A {
    void sayHello() {
        System.out.println("Hello from B");
    }
}

class C extends A {
    void sayHello() {
        System.out.println("Hello from C");
    }
}

// ✕ If Java supported:
// class D extends B, C { ... } => Causes ambiguity error!

```

However, **Java interfaces** (with default methods) allow multiple inheritance.

In case of method conflict, the implementing class **must override** and resolve it manually using `InterfaceName.super.method()`

```
interface A {  
    1 override  
    default void sayHello() {  
        System.out.println("Hello from Interface A");  
    }  
}  
  
1 implementation  
interface B {  
    1 override  
    default void sayHello() {  
        System.out.println("Hello from Interface B");  
    }  
}  
  
class C implements A, B {  
    // ✗ Must override to resolve conflict  
    public void sayHello() {  
        A.super.sayHello(); // Or B.super.sayHello();  
    }  
}
```

An interface in Java is like a **completely abstract class**:

- It defines **what a class must do**, but **not how** it does it.
- It contains **abstract methods** (by default), and optionally **default** or **static** methods (Java 8+).
- Classes that implement an interface must **provide definitions** for all its methods.

Use an interface when:

- You want to **define a contract** without any implementation.
- You want to support **multiple inheritance** (Java doesn't allow class multiple inheritance, but interfaces allow it).

- You want to separate "what" from "how" completely.

```
2 implementations
interface Animal {
    2 implementations
    void makeSound(); // abstract method
    2 implementations
    void eat();
}
```

```
// Dog implements Animal
class Dog implements Animal {
    public void makeSound() {
        System.out.println("Bark!");
    }

    public void eat() {
        System.out.println("Dog eats bones.");
    }
}

// Cat implements Animal
class Cat implements Animal {
    public void makeSound() {
        System.out.println("Meow!");
    }

    public void eat() {
        System.out.println("Cat eats fish.");
    }
}
```

## Interface in Real Life (Payment)

```
interface PaymentMethod {  
    2 implementations  
    void pay(double amount);  
}  
  
class CreditCard implements PaymentMethod {  
    public void pay(double amount) {  
        System.out.println("Paid $" + amount + " with credit card.");  
    }  
}  
  
class PayPal implements PaymentMethod {  
    public void pay(double amount) {  
        System.out.println("Paid $" + amount + " via PayPal.");  
    }  
}
```

```
public class Main {  
    public static void main(String[] args) {  
        PaymentMethod method = new CreditCard(); // Interface type  
        method.pay( amount: 100.00);  
  
        method = new PayPal(); // Swap at runtime  
        method.pay( amount: 75.50);  
    }  
}
```

## III. Dependency injection (Advanced)

### 3.1 What is "Coupling"?

In OOP, **coupling** refers to the **degree of direct knowledge** one class has about another.

- If a class is **heavily dependent** on another class, they are **tightly coupled**.
- If a class only depends on **abstractions (interfaces)** or **injected dependencies**, it is **loosely coupled**.

## 3.2 Tight Coupling – Explained

### Definition:

When a class **directly creates and controls** the lifecycle of its dependencies using `new`, it is tightly coupled.

### Problems:

- **Hard to test** (can't substitute a dependency easily)
- **Hard to change** (modifying one class may break others)
- **No flexibility** (always uses specific implementations)
- **Violates OOP principles** like **Open/Closed** and **Dependency Inversion**

```
class MySQLDatabase {
    public void connect() {
        System.out.println("Connected to MySQL");
    }
}

class Application {
    private MySQLDatabase db;

    public Application() {
        db = new MySQLDatabase(); // Tight coupling
    }

    public void start() {
        db.connect();
        System.out.println("App started");
    }
}
```

### Problems here:

- Can't replace `MySQLDatabase` with `PostgresDatabase` without modifying `Application`
- Hard to mock `MySQLDatabase` in testing

### 3.3 Loose Coupling – Explained

#### Definition:

When a class **depends on interfaces or abstractions**, and dependencies are **injected**, it is loosely coupled.

#### Benefits:

- **Flexible:** Swap dependencies easily
- **Testable:** Use mocks/stubs in unit tests
- **Maintainable:** Update modules without touching other parts
- **Extensible:** Add new features without breaking existing code
- Follows **SOLID principles**: especially **D** (Dependency Inversion)

```
interface Database {  
    // 2 implementations  
    void connect();  
}  
  
class MySQLDatabase implements Database {  
    public void connect() {  
        System.out.println("Connected to MySQL");  
    }  
}  
  
class PostgresDatabase implements Database {  
    public void connect() {  
        System.out.println("Connected to PostgreSQL");  
    }  
}
```



```

class Application {
    private Database db;

    // Constructor Injection - loose coupling
    public Application(Database db) {
        this.db = db;
    }

    public void start() {
        db.connect();
        System.out.println("App started");
    }
}

```

```

public class Main {
    public static void main(String[] args) {
        Database db = new MySQLDatabase(); // or new PostgresDatabase()
        Application app = new Application(db); // Inject dependency
        app.start();
    }
}

```

### 3.4 Inversion of Control

**Inversion of Control (IoC)** is a design principle where the control of **object creation** and **dependency management** is **inverted** from your code to a **container or framework**.

**In simple terms:**

Normally, your code controls when and how objects are created (new). With **IoC**, that control is handed over to something else (e.g., Spring).

Your class **does not create dependencies** — they are **injected from the outside**.

#### IoC vs Dependency Injection

- **IoC** is a **principle**
- **Dependency Injection (DI)** is a **pattern** (a way to achieve IoC)

So, DI is **one way** to implement IoC (the most popular one).

Let's Revisit the Example – WITHOUT IoC

```
class MySQLDatabase {
    public void connect() {
        System.out.println("Connected to MySQL");
    }
}

class Application {
    private MySQLDatabase db;

    public Application() {
        db = new MySQLDatabase(); // You control how the dependency is created (NO IoC)
    }

    public void start() {
        db.connect();
        System.out.println("App started");
    }
}
```

### Tight Coupling + No IoC

- The Application class **creates its own dependency**
- **YOU** control the flow and lifecycle

Now WITH IoC + DI

```
interface Database {
    // 1 implementation
    void connect();
}

class MySQLDatabase implements Database {
    public void connect() {
        System.out.println("Connected to MySQL");
    }
}
```

```

class Application {
    private Database db;

    // Constructor Injection - DI
    public Application(Database db) {
        this.db = db;
    }

    public void start() {
        db.connect();
        System.out.println("App started");
    }
}

```

```

public class Main {
    public static void main(String[] args) {
        // IoC happens here: we control which implementation to inject
        Database db = new MySQLDatabase();
        Application app = new Application(db);
        app.start();
    }
}

```

## ***Exercise:***

### ***Ex1. Game: The Prince Rescues the Princess***

#### **Scenario:**

The Princess is imprisoned by a Witch in a fortified castle that has **N layers of gates**. To enter the castle and rescue the Princess, the **Prince must pass through all these gates**.

At each gate, there is a gatekeeper. There are **3 types of gates**:

#### **Gate Types:**

##### **1. Business Gate**

- **Gatekeeper:** A merchant.
- **Condition to pass:** The Prince must **buy goods** from the merchant.

- The amount of money required is:  
money = unit price \* quantity of goods.

## 2. Academic Gate

- **Gatekeeper:** A philosopher.
- **Condition to pass:** The Prince must **answer a question** from the philosopher.
- Each question has an **intelligence index**.
- The Prince's **intelligence must be greater than or equal to** the required index to pass.
- **Note:** After answering the question, the Prince **does not lose** intelligence.

## 3. Power Gate

- **Gatekeeper:** A warrior.
- **Condition to pass:** The Prince must **defeat the warrior** in combat.
- Each warrior has a **power index**.
- The Prince's **power must be greater than or equal to** the warrior's power to win.
- **After winning,** the Prince's **power is reduced by the warrior's power index**.

### Summary Table of Gate Conditions:

| Gate Type     | Gatekeeper  | Condition to Pass                                                       |
|---------------|-------------|-------------------------------------------------------------------------|
| Business Gate | Merchant    | Money lost = unit price * quantity                                      |
| Academic Gate | Philosopher | Prince's intelligence $\geq$ question's intelligence                    |
| Power Gate    | Warrior     | Prince's power $\geq$ warrior's power<br>Then lose that amount of power |

## Programming Task using Object-Oriented Programming Concepts

(Using **Inheritance** and **Polymorphism**)

### Design Requirements:

- **Design a class diagram** that models the gate system above.
- **Build a program** that fulfills the following functionalities:

**Functional Requirements:**

- Input** the list of **N gates** of the castle.
- Input** the Prince's initial stats:

- Amount of money
- Intelligence index
- Power index

The program should **determine** whether the Prince can **successfully pass through all the gates** and **rescue the Princess** based on the given stats.

***Ex2. Software Company Employee Salary Management***

The software development company Alpha needs to manage its office employees, which include:

- Manager
- Programmer
- Designer
- Tester

**All employees have the following basic attributes:**

- Employee ID
- Full Name
- Date of Birth
- Address
- Salary Coefficient
- Start Date of Employment

**Salary Calculation Rules for each type of employee:**

- **Manager:**  
Salary = Base Salary × Salary Coefficient

- **Programmer:**

Salary = Base Salary × Salary Coefficient + Overtime Pay

- **Designer:**

Salary = Base Salary × Salary Coefficient + Bonus

- **Tester:**

Salary = Base Salary × Salary Coefficient + Error × 200,000

(where Error is the number of critical bugs found in the software by the employee that month)

### Object-Oriented Programming (OOP) Requirements:

Apply inheritance and polymorphism principles to write a program to implement the following features:

- (1) Input a list of N employees in the company (N is entered from the keyboard).
- (2) Calculate the total salary that the company needs to pay to all employees in a month.

### Ex3. Blood Type Inheritance and Compatibility Program

#### Background:

In the early 1900s, based on the presence of antigens on the surface of red blood cells, scientists identified that humans have **four different blood types: O, A, B, and AB**.

This classification system, known as the **ABO blood group system**, provides doctors with crucial information to **choose the appropriate blood type for transfusion** and also **predict the potential blood type of a child based on the parents' blood types**, using the principles of genetics.

#### Child's Blood Type from Parents:

- Given the **blood types of the father and the mother**, one can determine the **possible blood types of the child** based on genetic inheritance.

|                   |    | Nhóm máu người cha |                 |              |          |                                     |
|-------------------|----|--------------------|-----------------|--------------|----------|-------------------------------------|
|                   |    | A                  | B               | AB           | O        |                                     |
| Nhóm máu người mẹ | A  | A hoặc O           | A, B, AB hoặc O | A, B hoặc AB | A hoặc O | Dự đoán khả năng nhóm máu người con |
|                   | B  | A, B, AB hoặc O    | B hoặc O        | A, B hoặc AB | B hoặc O |                                     |
|                   | AB | A, B hoặc AB       | A, B hoặc AB    | A, B hoặc AB | A hoặc B |                                     |
|                   | O  | A hoặc O           | B hoặc O        | A hoặc B     | O        |                                     |

## **Rhesus (Rh) Factor:**

In addition to the ABO system, there is also the **Rh (Rhesus) blood group system**.

- Red blood cells may carry a **protein called the Rhesus (D antigen)**.
- If present, the person is **Rh positive (Rh+)**.
- If absent, the person is **Rh negative (Rh-)**.

Thus, the full blood types become:

**A-, B-, O-, AB-** for Rh-negative types.

- **Rh-negative blood types** are rare, accounting for only about **0.04% of the global population**.
- People with **Rh-negative blood** can **only receive and donate to other Rh-negative people**.
- Especially for **Rh-negative women**, having children can be risky, as the fetus could suffer complications or death if Rh incompatibility is not managed.

## **Blood Donation Compatibility Rules:**

- **Rh+ individuals** can **donate only to other Rh+ people**, but they can **receive from both Rh+ and Rh- donors**.
- **Rh- individuals** can **donate to both Rh+ and Rh-**, but they can **only receive blood from Rh- donors**.

**Important:** If an **Rh- person** receives **Rh+ blood for the first time**, there may not be an immediate reaction. However, a **second Rh+ transfusion can result in severe complications** due to blood transfusion reactions.

The same applies to **Rh- mothers having Rh+ children** in multiple pregnancies.

Khả năng tương thích: ✓: Có thể cho - nhận.

✗: Không thể cho - nhận.

| Bảng khả năng tương thích hồng cầu |           |    |    |    |    |    |     |     |
|------------------------------------|-----------|----|----|----|----|----|-----|-----|
| Người nhận                         | Người cho |    |    |    |    |    |     |     |
|                                    | O-        | O+ | A- | A+ | B- | B+ | AB- | AB+ |
| O-                                 | ✓         | ✗  | ✗  | ✗  | ✗  | ✗  | ✗   | ✗   |
| O+                                 | ✓         | ✓  | ✗  | ✗  | ✗  | ✗  | ✗   | ✗   |
| A-                                 | ✓         | ✗  | ✓  | ✗  | ✗  | ✗  | ✗   | ✗   |
| A+                                 | ✓         | ✓  | ✓  | ✓  | ✗  | ✗  | ✗   | ✗   |
| B-                                 | ✓         | ✗  | ✗  | ✗  | ✓  | ✗  | ✗   | ✗   |
| B+                                 | ✓         | ✓  | ✗  | ✗  | ✓  | ✓  | ✗   | ✗   |
| AB-                                | ✓         | ✗  | ✓  | ✗  | ✓  | ✗  | ✓   | ✗   |
| AB+                                | ✓         | ✓  | ✓  | ✓  | ✓  | ✓  | ✓   | ✓   |

## Programming Requirements

(Use **Object-Oriented Programming** concepts such as **inheritance** and **polymorphism**)

### Tasks:

Implement a program that performs the following:

- Input a list of people's blood types.
- Given the blood types of 3 people (father, mother, and child), determine whether the child's blood type is genetically valid.
- Choose one person X from the list. List all other people who can donate blood to person X.

### *Ex4. Big Five Model – Psychological Trait Assessment*

The **Big Five Model** is a psychological framework developed and researched by scientists around the world. This model suggests that every individual's personality is composed of five key factors, and psychological assessments aim to determine the degree to which each factor is expressed. The Big Five Model is highly regarded for its **reliability and scientific basis**.





The Big Five personality traits are:

1. Conscientiousness (C)
2. Extraversion (E)
3. Agreeableness (A)
4. Openness to Experience (O)
5. Neuroticism (N)

Hence, it is also known by the acronym **OCEAN**.

### Detailed Description of Each Trait

| Trait                             | High Score                                                                     | Low Score                                                                |
|-----------------------------------|--------------------------------------------------------------------------------|--------------------------------------------------------------------------|
| <b>Openness to Experience (O)</b> | Enjoys new ideas, curious, independent, creative, embraces change and novelty. | Conservative, resistant to new ideas, prefers stability and routine.     |
| <b>Conscientiousness (C)</b>      | Hardworking, disciplined, responsible, persistent, loyal to organizations.     | Disorganized, easily gives up, lacks responsibility and self-discipline. |
| <b>Extraversion (E)</b>           | Outgoing, energetic, sociable, enthusiastic, enjoys being around people.       | Shy, reserved, prefers to work alone, avoids social interaction.         |
| <b>Agreeableness (A)</b>          | Friendly, empathetic, cooperative, altruistic, modest.                         | Competitive, self-centered, less empathetic and cooperative.             |

|                        |                                                             |                                                            |
|------------------------|-------------------------------------------------------------|------------------------------------------------------------|
| <b>Neuroticism (N)</b> | Emotionally unstable, anxious, moody, vulnerable to stress. | Emotionally stable, calm, resilient, unaffected by stress. |
|------------------------|-------------------------------------------------------------|------------------------------------------------------------|

Each trait is further broken down into 6 sub-factors (not detailed here for brevity).

### Big Five Personality Test Result Format

Example result: O93-C74-E31-A96-N5

This means:

- **O93**: 93% of people scored lower than you in **Openness** → You are highly open to new experiences and ideas.
- **C74**: 74% of people scored lower than you in **Conscientiousness** → You are quite organized and self-disciplined.

This test is widely used in **recruitment and HR evaluation** to assess candidates and anticipate potential risks in hiring.

### Risk Profiles

Certain patterns should be flagged by recruiters or organizations:

- **(a)** High C → Rule-abiding, hardworking, persistent.
- **(b)** Low C → Disorganized, irresponsible, careless, impulsive.
- **(c)** High N → Emotionally unstable, high stress levels, low commitment.
- **(d)** Low E + High N → Difficulty seeking or processing new information, lacks important career-related skills.

These patterns (b, c, d) are **risk indicators**.

### Score Thresholds

- Scores  $\geq 70$  → **High**
- Scores  $\leq 30$  → **Low**
- Scores in between → **No clear personality direction identified**

### Programming Task (Object-Oriented Design)

Using **Object-Oriented Programming (OOP)** principles such as **inheritance** and **polymorphism**, complete the following:

**Write a program that performs the following:**

1. **Input** psychological assessment results for **one person**.
2. **Input** assessment results for **n people** in a list.
3. **Select a person** from the list and **display their personality description** based on their scores.

**Example input:**

O70-C30-E60-A96-N10

**Expected output:**

70 Openness (O): High – Enjoys new ideas, curious, independent, creative...

30 Conscientiousness (C): Low – Disorganized, gives up easily, lacks discipline...

60 Extraversion (E): No specific conclusion

96 Agreeableness (A): High – Friendly, empathetic, cooperative...

10 Neuroticism (N): Low – Emotionally stable, handles stress well...

4. **Identify people at high risk**, based on patterns (b, c, d) above (1 point)